



**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»**

Институт: **ИВТИ**

Кафедра: **ВМСС**

Направление подготовки:

**09.03.01 Информатика и вычислительная
техника**

**Отчёт по дисциплине “Технология программирования”
на тему “Разработка проекта для администрирования музыкального
магазина”**

Работу выполнил

Студент Балныкин Климентий

Группа А-07-24

Работу проверил

Щёголев Павел

Москва, 2025

Оглавление

Введение	3
1. Техническое задание	4
2. Разработка диаграммы классов	8
3. Разработка и реализация программы на языке C++	11
3.1 Разработка пользовательского интерфейса	11
3.2 Описание программы	13
3.3 Тестирование программы	14
4 Разработка и реализация программы на языке C#	19
4.1 Разработка пользовательского интерфейса	19
4.2 Описание программы	21
4.3 Тестирование программы	22
5. Сравнительный анализ реализаций	26
Заключение	28

Введение

В сфере розничной торговли музыкальными инструментами эффективное управление ассортиментом, учёт продаж и контроль поставок являются ключевыми факторами, обеспечивающими стабильную работу торгового предприятия. Ручное ведение учёта товаров и операций может приводить к ошибкам, потере актуальных данных и снижению качества обслуживания клиентов. В связи с этим актуальной является задача автоматизации процессов управления складом, продажами и формированием отчётности.

Цель работы: разработка приложения для автоматизации управления музыкальным магазином.

Задачи работы:

- Анализ предметной области и определение функциональных требований к системе управления музыкальным магазином.
- Проектирование объектно-ориентированной структуры приложения с использованием классов для представления товаров, продаж, поставок и управляющей логики.
- Реализация модулей для ведения каталога товаров, оформления продаж и регистрации поставок.
- Реализация файлового хранения данных для сохранения информации о товарах, продажах и поставках.
- Тестирование работы приложения в различных режимах эксплуатации.

Разрабатываемая программа обеспечивает комплексное управление деятельностью музыкального магазина, начиная с учёта ассортимента и заканчивая формированием отчётных данных по продажам и поставкам.

1. Техническое задание

Общие сведения

Тема: Симулятор работы кассы музыкального магазина.

Название проекта: «MusicStore»

Цель: Программа предназначена для сотрудников музыкального магазина.

Система позволяет вести каталог товаров, оформлять заказы покупателям в магазине, оформлять поставки и формировать финансовые и складские отчёты.

Целевая аудитория: администратор музыкального магазина.

Описание проекта

Программа моделирует внутреннюю работу розничного музыкального магазина.

Основные задачи:

- учёт ассортимента и остатков товаров,
- оформление заказов,
- приём и учёт поставок со склада,
- формирование отчётности по заказам и поставкам.

Функциональные требования

Каталог товаров

Просмотр списка товаров по категориям (в конце ТЗ указан весь ассортимент).

Изменение данных существующего товара (цена).

Заказы

Создание заказа:

Выбор товаров

Добавление в корзину.

Оплата

Типы оплаты: наличные / банковская карта.

Наличные - сообщение об успешной оплате

По карте: ввод номера карты (форматная проверка, указано ниже) - сообщение об успешной оплате.

Поставка товаров

Регистрация поставки (не более 15 наименований товаров).

Для каждого наименования можно указать количество не более 100 (целое ≥ 0).

Автоматическое увеличение остатков в каталоге.

Отчёты

Отчёты выводятся по всем записям в порядке добавления.

Заказы за весь период (ID заказа /n, список «товар, кол-во» /n, общая сумма заказов).

История поставок за выбранный период (ID поставки /n, список «товар, кол-во»).

Основные сценарии использования

Администратор запускает программу

Заказ товара

Администратор выбирает «Новый заказ» → выбирает товар и кол-во → программа выводит ID заказа, фиксирует заказ в отчете, рассчитывает и выводит сумму → выбирает способ оплаты (если карта, необходимо ввести номер карты, проверка до тех пор, пока номер карты не будет соответствовать шаблону) → сообщает об успешной оплате.

категории → номер категории → список товаров → номер товара → количество → добавить ещё товары/завершить

Создание оптовой поставки

Администратор выбирает «Новая поставка» → выбирает товар и кол-во → программа увеличивает кол-во у выбранных товаров и фиксирует поставку в отчете.

категории → номер категории → список товаров → номер товара → количество → добавить ещё товары/завершить

Управление каталогом

Администратор выбирает «Каталог» → просматривает все категории и товары в них / изменяет цену товара. Если выбирает изменить, то нужно выбрать товар, далее ввести новую цену

Формирование отчёта

Администратор выбирает «Отчёты» → запрашивает отчет (выбирает заказ или поставки) → получает файл соответственного отчёта.

Требования к данным

Товар: категория, название, цена, количество.

Заказ: ID, список товаров, способ оплаты, итоговая сумма.

Поставка: ID, список товаров и их количество.

Форматы данных:

Банковская карта: «1111888811118888», 16 цифр(0-9) без пробелов

Номер телефона: «89012345678», 11 цифр(0-9) без пробелов

ID: «000001», 6 цифр без пробелов

Каталог товаров:

Товары (струнные, клавишные, духовые, ударные инструменты, аксессуары):

Гитары:

Классическая гитара – 12 000 Р

Акустическая гитара – 15 000 Р

Бас-гитара – 18 000 Р

Электрогитара – 20 000 Р

Клавишные инструменты:

Фортепиано – 150 000 Р

Синтезатор – 35 000 Р

Аккордеон – 45 000 Р

Духовые инструменты:

Флейта – 12 000 Р

Кларнет – 18 000 Р

Саксофон – 55 000 Р

Ударные инструменты:

Барабанная установка – 60 000 Р

Малый барабан – 10 000 Р

Бонго – 8 000 Р

Аксессуары:

Струны для гитары – 700 Р

Медиаторы – 100 Р

Чехлы – 1 800 Р

Кабели и переходники – 500 Р

Тюнеры – 1 000 Р

2.Разработка диаграммы классов

Была разработана диаграмма классов (рис 1).

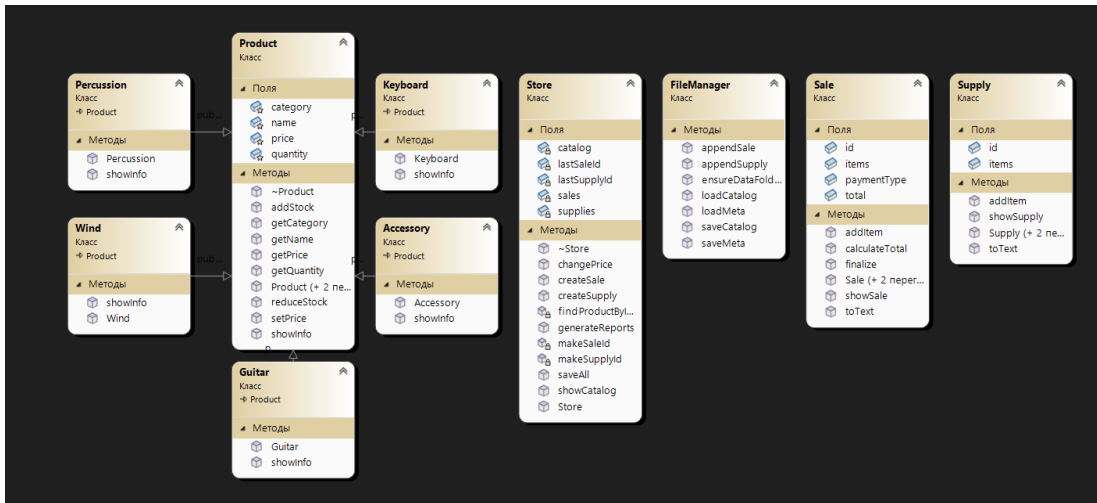


Рис. 1. Диаграмма классов

Класс Product

Назначение: Базовый класс, представляющий товар музыкального магазина

Табл. 1

Описание полей и методов класса Product

Описание полей		
Имя	Модификатор доступа	Описание
std::string name	protected	Название товара
category	protected	Категория товара
price	protected	Цена товара
quantity	protected	Количество на складе
Описание методов		
Имя	Модификатор доступа	Описание
Product()	public	Конструктор по умолчанию
Product(const Product&)	public	Конструктор копирования
Product(Product&&)	public	Конструктор перемещения
~Product()	public	Виртуальный деструктор
getName()	public	Возвращает название товара
getCategory()	public	Возвращает категорию товара
getPrice()	public	Возвращает цену товара
getQuantity()	public	Возвращает количество товара
setPrice()	public	Изменяет цену товара
addStock()	public	Увеличивает количество товара
reduceStock()	public	Уменьшает количество товара
showInfo()	public	Выводит информацию о товаре

Класс Store

Назначение: Основной управляющий класс магазина

Табл. 2

Описание полей и методов класса Store

Описание полей		
Имя	Модификатор доступа	Описание
catalog	private	Название товара
sales	private	Категория товара
supplies	private	Цена товара
lastSaleId	private	
lastSupplyId	private	Количество на складе
Описание методов		
Имя	Модификатор доступа	Описание
Store()	public	Конструктор по умолчанию
~Store()	public	Деструктор
showCatalog()	public	Отображение каталога товаров
createSale()	public	Создание новой продажи
createSupply()	public	Создание новой поставки
changePrice()	public	Изменение цены товара
generateReports()	public	Формирование отчётов
saveAll()	public	Сохранение всех данных

Класс Sale

Назначение: Класс, представляющий продажу товаров

Табл. 3

Описание полей и методов класса Sale

Описание полей		
Имя	Модификатор доступа	Описание
id	public	Идентификатор продажи
items	public	Список проданных товаров
total	public	Итоговая сумма
paymentType	public	Тип оплаты
Описание методов		
Имя	Модификатор доступа	Описание
Sale()	public	Конструктор по умолчанию
addItem()	public	Добавление товара в продажу
calculateTotal()	public	Расчёт итоговой суммы
finalize()	public	Завершение продажи
showSale()	public	Отображение информации о продаже

toText()	public	Сериализация продажи в текст
----------	--------	------------------------------

Класс Supply

Назначение: Класс, представляющий поставку товаров

Табл. 4

Описание полей и методов класса Supply

Описание полей		
Имя	Модификатор доступа	Описание
id	public	Идентификатор поставки
items	public	Список товаров поставки
Описание методов		
Имя	Модификатор доступа	Описание
Supply()	public	Конструктор по умолчанию
addItem()	public	Добавление товара в поставку
showSupply()	public	Отображение информации о поставке
toText()	public	Сериализация поставки в текст

Класс FileManager

Назначение: Служебный класс для работы с файловым хранилищем

Табл. 5

Описание полей и методов класса FileManager

Описание методов		
Имя	Модификатор доступа	Описание
ensureDataFolder()	public static	Создание каталога data
saveCatalog()	public static	Сохранение каталога
loadCatalog()	public static	Загрузка каталога
saveMeta()	public static	Сохранение метаданных
loadMeta()	public static	Загрузка метаданных
appendSale()	public static	Добавление продажи в файл
appendSupply()	public static	Добавление поставки в файл

3. Разработка и реализация программы на языке C++

3.1 Разработка пользовательского интерфейса

Была разработана пользовательская консоль (рис. 2).

```
=== Музыкальный магазин (админ) ===
1. Просмотр каталога
2. Создать заказ (продажа)
3. Создать поставку
4. Изменить цену товара
5. Сформировать отчёт (файлы в data/)
0. Сохранить и выйти
Выбор:
```

Рис. 2. Пользовательская консоль

1) Просмотр каталога

Функция выводит в консоль каталог товаров (рис. 3).

```
=== Каталог ===
1. Классическая гитара [Гитары] – 12000 Р, в наличии: 26
2. Акустическая гитара [Гитары] – 15000 Р, в наличии: 84
3. Бас-гитара [Гитары] – 18000 Р, в наличии: 10
4. Электрогитара [Гитары] – 20000 Р, в наличии: 10
5. Фортепиано [Клавишные] – 150000 Р, в наличии: 10
6. Синтезатор [Клавишные] – 35000 Р, в наличии: 50
7. Аккордеон [Клавишные] – 45000 Р, в наличии: 8
8. Флейта [Духовые] – 11000 Р, в наличии: 10
9. Кларнет [Духовые] – 18000 Р, в наличии: 10
```

Рис. 3. Каталог товаров

2) Создать заказ (продажа).

При выборе этой функции программа сперва просит ввести номер товара, затем – количество. Если введенное пользователем кол-во товаров имеется на складе, то выводится сообщение об успешном добавлении товаров в заказ (рис. 4).

```
Введите номер товара (0 - завершить):1  
  
В наличии: 26. Введите количество для добавления в заказ:3  
  
Добавлено: Классическая гитара x3
```

Рис. 4. Оформление заказа

После добавления товаров в корзину, их нужно оплатить (рис. 5).

```
Итоговая сумма: 36000 Р  
Выберите тип оплаты: 1 - Наличные, 2 - Карта:1
```

Рис. 5. Оплата товаров

3) Создать поставку

При выборе этой функции программа сперва просит ввести номер товара, затем – количество. После этого выводится сообщение об успешном добавлении товаров в поставку (рис. 6).

```
Введите номер товара для поставки (0 - завершить):3  
  
Введите количество (целое, <=100):10  
  
Поставлено: Бас-гитара x10
```

Рис. 6. Оформление поставки

4) Изменить цену товара

Эта функция позволяет изменить цену товара, сперва необходимо ввести номер товара, затем программа выводит текущую цену и просит ввести новую, после чего выводит сообщение об успешном изменении цены (рис.7).

```
Введите номер товара для изменения цены:16  
  
Текущая цена: 1800 Р. Введите новую цену:2660  
  
Цена изменена.
```

Рис. 7. Изменение цены товара

5) Сформировать отчет.

Эта функция формирует отчет по продажам и закупкам в отдельные файлы и выводит соответствующее сообщение (рис. 8).

```
=== Отчёты ===  
История продаж: data/sales.txt  
История поставок: data/supplies.txt  
Текущий каталог/остатки: data/catalog.txt  
Открыть эти файлы в папке data.|
```

Рис. 8. Формирование отчета

Пример файла, в который выводится отчет (рис. 9).

```
ID:000001  
ITEMS:Акустическая гитара;8,Барабанная установка;5  
TOTAL:420000  
PAYMENT:Наличные  
----  
ID:000002  
ITEMS:Классическая гитара;1,Акустическая гитара;1  
TOTAL:27000  
PAYMENT:Карта  
----  
ID:000003  
ITEMS:Классическая гитара;2  
TOTAL:24000  
PAYMENT:Карта
```

Рис. 9. Пример файла с отчетом

3.2 Описание программы

Общее описание

Программа представляет собой систему управления для музыкального магазина, реализованную на C++. Она позволяет администратору управлять каталогом музыкальных инструментов и аксессуаров, обрабатывать продажи, регистрировать поставки товаров и генерировать отчеты.

Основные функции:

- Управление каталогом товаров
- Обработка продаж
- Управление поставками

- Работа с данными
- Файловая система

В программе реализованы основные принципы объектно-ориентированного программирования: полиморфизм, наследование и инкапсуляция.

Реализация по чек-листу

Табл. 6

Таблица соответствия программы и чек-листа

Пункт чек-листа	Пример реализации в коде
Класс	<code>class Sale, class Supply, class Store, class FileManager</code>
Геттер	<code>std::string Product::getName() const { return name; }</code>
Сеттер	<code>void Product::setPrice(double newPrice) { price = newPrice; }</code>
Конструктор с параметрами	<code>Product::Product(const std::string &name_, const std::string &category_, double price_, int quantity_) : name(name_), category(category_), price(price_), quantity(quantity_) { }</code>
Конструктор копирования	<code>Sale::Sale(const Sale& other) : id(other.id), items(other.items), total(other.total), paymentType(other.paymentType) { }</code>
Конструктор без параметров	<code>Supply(const std::string& id = "");</code>
Конструктор перемещения	<code>Sale::Sale(Sale&& other) noexcept: id(std::move(other.id)), items(std::move(other.items)), total(other.total), paymentType(std::move(other.paymentType)) { other.total = 0.0; }</code>
Деструктор	<code>Product::~~Product() = default;</code>
Виртуальный деструктор	<code>virtual ~Product();</code>
Дружественные функции	<code>friend std::ostream &operator<<(std::ostream &os, const Product &product);</code>
Перегрузка ввода и вывода	<code>std::ostream &operator<<(std::ostream &os, const Product &p);</code>
Одиночное наследование	<code>class Guitar : public Product {</code>
Агрегация	<code>std::vector<Product*> catalog; // агрегация - продукты существуют независимо от магазина</code>
Ассоциация	<code>std::vector<std::pair<std::string, int>> items;</code>
Зависимость	<code>#include "product.h" // зависимость</code>
Раннее и позднее связывание	<code>virtual void showInfo() const; // в базовом классе Product</code>

3.3 Тестирование программы

Объект испытаний – разработанная программа системы управления складскими ячейками.

Цель испытаний:

- Проверка корректности работы функций программы
- Выявление ошибок и несоответствий в логике работы программы
- Проверка обработки граничных условий и исключительных ситуаций
- Оценка надежности системы управления памятью

Средства испытаний - среда разработки: Visual Studio Code / CLion, компилятор C++ с поддержкой стандарта C++11 или выше, командная строка для запуска программы.

Порядок и методы испытаний:

Функциональное тестирование ("черный ящик"), принцип: тестирование без знания внутренней структуры программы, проверка только входных и выходных данных.

Порядок тестирования:

- Нормальные условия — штатные сценарии работы.
- Граничные условия — работа с минимальными/максимальными значениями, заполненными/пустыми ячейками.
- Исключительные условия — ввод некорректных данных, попытки выполнить невозможные операции.

Тестирование в нормальных условиях

Тест 1.1. Просмотр каталога при запуске программы.

Ожидаемый результат: Отображение загруженного каталога из файла или создание каталога по умолчанию.

Фактический результат: Каталог успешно загружается (рис. 10)

```

=== Каталог ===
1. Классическая гитара [Гитары] — 12000 Р, в наличии: 26
2. Акустическая гитара [Гитары] — 15000 Р, в наличии: 84
3. Бас-гитара [Гитары] — 18000 Р, в наличии: 10
4. Электрогитара [Гитары] — 20000 Р, в наличии: 10
5. Фортепиано [Клавишные] — 150000 Р, в наличии: 10
6. Синтезатор [Клавишные] — 35000 Р, в наличии: 50
7. Аккордеон [Клавишные] — 45000 Р, в наличии: 8
8. Флейта [Духовые] — 11000 Р, в наличии: 10
9. Кларнет [Духовые] — 18000 Р, в наличии: 10

```

Рис. 10. Каталог

Тест 1.2. Создание продажи с одним товаром

Ожидаемый результат: Успешное создание продажи, уменьшение остатков товара.

Фактический результат: Продажа создана успешно (рис. 11)

```
Итоговая сумма: 24000 Р
Выберите тип оплаты: 1 - Наличные, 2 - Карта:1

ID продажи: 000006
- Классическая гитара x1
- Классическая гитара x1
Итого: 24000 Р
Оплата: Наличные
Оплата прошла успешно! Продажа сохранена.
```

Рис. 11. Оформление заказа

Тест 1.3. Создание поставки.

Ожидаемый результат: Успешное добавление товаров в поставку, увеличение остатков.

Фактический результат: Поставка создана успешно (рис. 12)

```
-----
Поставка зарегистрирована успешно!
ID поставки: 000006
Содержимое поставки:
- Классическая гитара x1
```

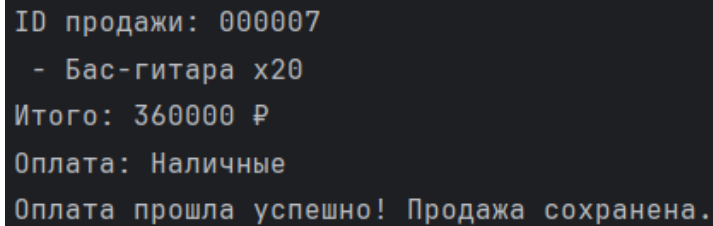
Рис. 12. Оформление поставки

Тестирование в граничных условиях

Тест 2.1. Продажа максимально возможного количества товара.

Ожидаемый результат: Продажа всего доступного количества товара (граничное значение).

Фактический результат: Продажа выполнена успешно (рис. 13)



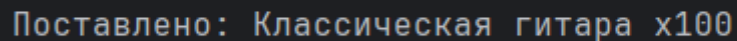
```
ID продажи: 000007
- Бас-гитара x20
Итого: 360000 Р
Оплата: Наличные
Оплата прошла успешно! Продажа сохранена.
```

Рис. 13. Успешное выполнение продажи

Тест 2.2. Поставка максимального разрешенного количества (100 единиц).

Ожидаемый результат: Успешное добавление 100 единиц товара в поставку.

Фактический результат: Поставка выполнена успешно (рис. 14)



```
Поставлено: Классическая гитара x100
```

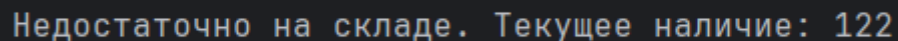
Рис. 14. Поставка максимального количества товара

Тестирование в исключительных условиях

Тест. 3.1. Попытка продажи при недостаточном количестве товара

Ожидаемый результат: Отказ в продаже с соответствующим сообщением.

Фактический результат: Выведено сообщение об ошибке (рис. 15)



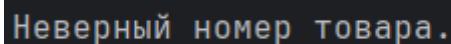
```
Недостаточно на складе. Текущее наличие: 122
```

Рис. 15. Сообщение об ошибке, неверное число товара

Тест 3.2 Ввод некорректного номера товара (отрицательного или больше размера каталога)

Ожидаемый результат: Сообщение об ошибке и повторный запрос.

Фактический результат: Выведено сообщение об ошибке (рис. 16)



```
Неверный номер товара.
```

Рис. 16. Ошибка, неверный номер товара

Тест 3.3 Ввод отрицательного количества при продаже

Ожидаемый результат: Сообщение об ошибке.

Фактический результат: Выведено сообщение об ошибке (рис. 17)

Количество должно быть положительным.

Рис. 17. Сообщение об ошибке, неверное число товара

Результаты тестирования приведены в таблице

Табл. 7

Результаты тестирования программы

Номер теста	Условия тестирования	Результат
1.1	Нормальные	Успешно
1.2	Нормальные	Успешно
1.3	Нормальные	Успешно
2.1	Граничные	Успешно
2.2	Граничные	Успешно
3.1	Исключительные	Успешно
3.2	Исключительные	Успешно
3.3	Исключительные	Успешно

4 Разработка и реализация программы на языке C#

4.1 Разработка пользовательского интерфейса

Было разработано пользовательское меню (рис. 18).

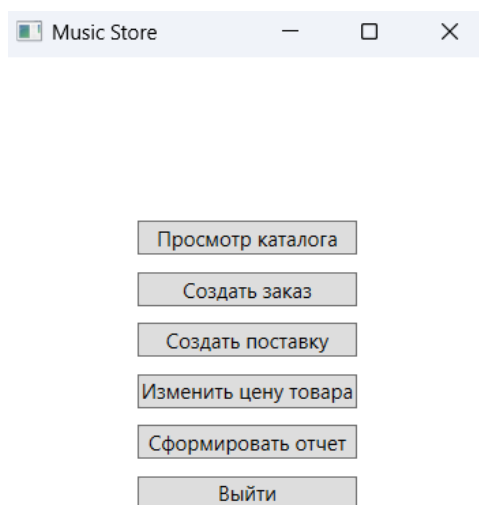


Рис. 18. Меню приложения

1) Просмотр каталога.

Эта функция выводит каталог (рис. 19).

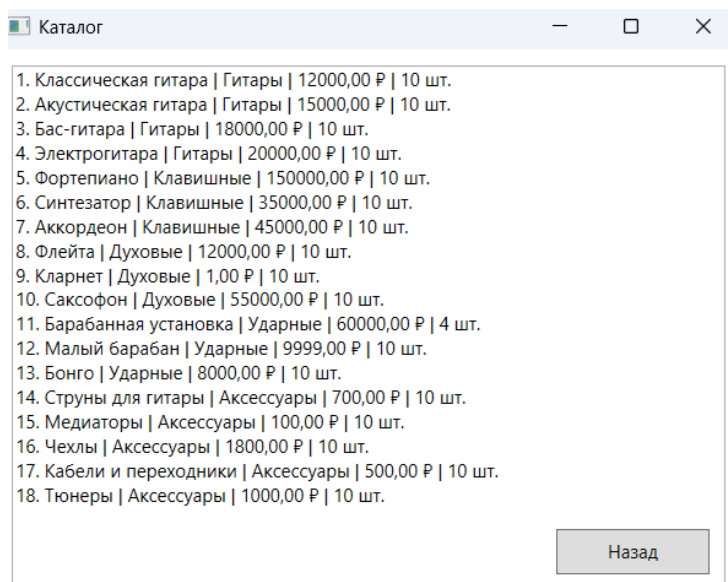
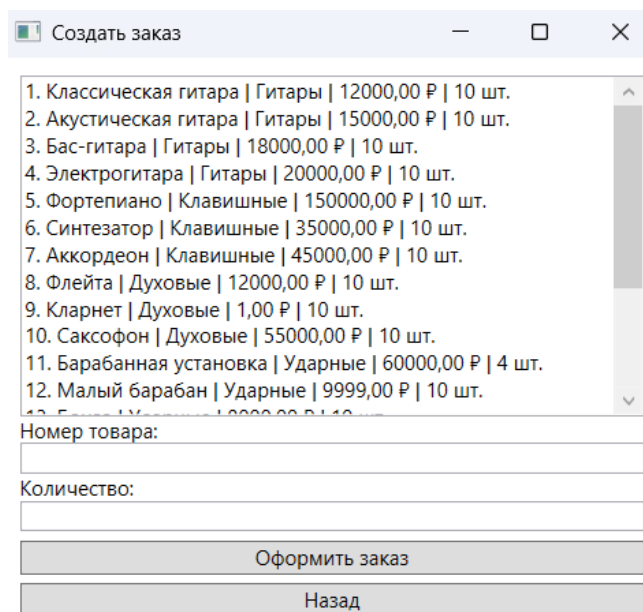


Рис. 19. Каталог

2) Создать заказ.

Эта функция просит ввести номер товара и кол-во (рис. 20).



Создать заказ

- 1. Классическая гитара | Гитары | 12000,00 Р | 10 шт.
- 2. Акустическая гитара | Гитары | 15000,00 Р | 10 шт.
- 3. Бас-гитара | Гитары | 18000,00 Р | 10 шт.
- 4. Электрогитара | Гитары | 20000,00 Р | 10 шт.
- 5. Фортепиано | Клавишные | 150000,00 Р | 10 шт.
- 6. Синтезатор | Клавишные | 35000,00 Р | 10 шт.
- 7. Аккордеон | Клавишные | 45000,00 Р | 10 шт.
- 8. Флейта | Духовые | 12000,00 Р | 10 шт.
- 9. Кларнет | Духовые | 1,00 Р | 10 шт.
- 10. Саксофон | Духовые | 55000,00 Р | 10 шт.
- 11. Барабанная установка | Ударные | 60000,00 Р | 4 шт.
- 12. Малый барабан | Ударные | 9999,00 Р | 10 шт.

Номер товара:

Количество:

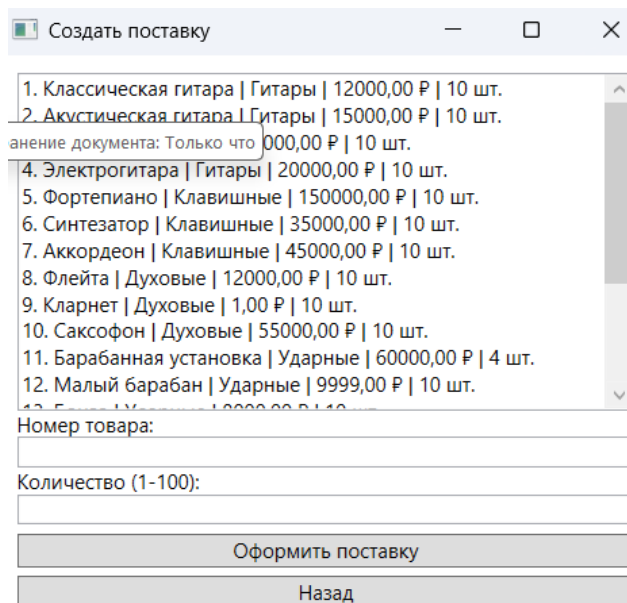
Оформить заказ

Назад

Рис. 20 Оформление заказа

3) Создать поставку.

Эта функция просит ввести номер товара и кол-во (1-100). После ввода этих данных создается поставка (рис. 21).



Создать поставку

анение документа: Только что

- 1. Классическая гитара | Гитары | 12000,00 Р | 10 шт.
- 2. Акустическая гитара | Гитары | 15000,00 Р | 10 шт.
- 3. Бас-гитара | Гитары | 18000,00 Р | 10 шт.
- 4. Электрогитара | Гитары | 20000,00 Р | 10 шт.
- 5. Фортепиано | Клавишные | 150000,00 Р | 10 шт.
- 6. Синтезатор | Клавишные | 35000,00 Р | 10 шт.
- 7. Аккордеон | Клавишные | 45000,00 Р | 10 шт.
- 8. Флейта | Духовые | 12000,00 Р | 10 шт.
- 9. Кларнет | Духовые | 1,00 Р | 10 шт.
- 10. Саксофон | Духовые | 55000,00 Р | 10 шт.
- 11. Барабанная установка | Ударные | 60000,00 Р | 4 шт.
- 12. Малый барабан | Ударные | 9999,00 Р | 10 шт.

Номер товара:

Количество (1-100):

Оформить поставку

Назад

Рис. 21. Оформление поставки

4) Изменить цену товара.

Эта функция позволяет изменить цену товара, сперва необходимо выбрать номер товара и указать новую цену (рис. 22).

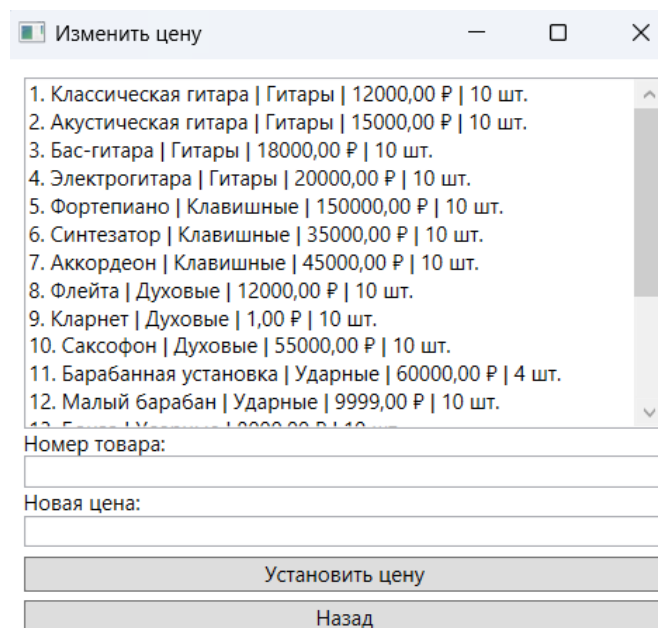


Рис. 22. Изменение цены товара

5) Сформировать отчет.

Функция формирует отчет в отдельных файлах (рис. 23).

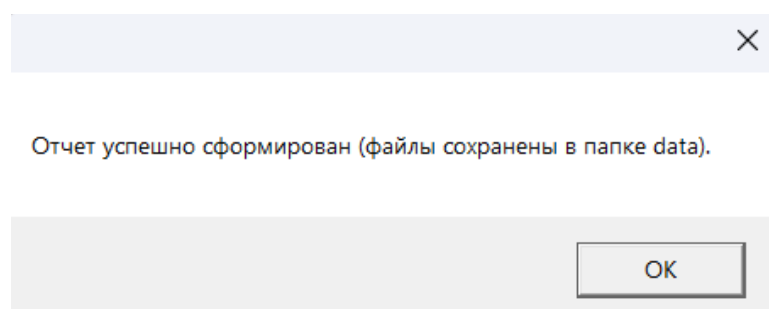


Рис. 23. Сообщение о формировании отчета

4.2 Описание программы

WPF-приложение для управления музыкальным магазином с графическим интерфейсом. Программа предоставляет функционал для управления каталогом товаров, оформления заказов (продаж), создания поставок и изменения цен.

Архитектура приложения

- **Главное окно (MainWindow):** Меню с навигацией по функциям
- **Окно каталога (CatalogWindow):** Просмотр товаров
- **Окно заказа (OrderWindow):** Оформление продаж
- **Окно поставки (SupplyWindow):** Пополнение товаров
- **Окно цены (PriceWindow):** Изменение цен товаров

- **Класс FileManager:** Управление сохранением/загрузкой данных
- **Классы данных:** Product, Sale, Supply

В программе реализованы основные принципы объектно-ориентированного программирования: полиморфизм, наследование и инкапсуляция.

4.3 Тестирование программы

Описание способа тестирования приведено в пункте 3.3

Тестирование в нормальных условиях

Тест 1.1 Загрузка каталога при запуске программы

Ожидаемый результат: Загрузка каталога из файла или создание стандартного каталога.

Фактический результат: Каталог успешно загружается/создается (рис. 24)

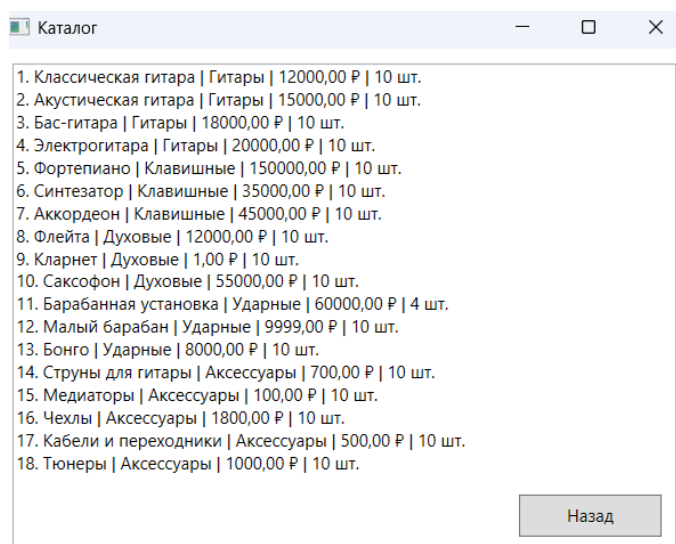


Рис. 24. Успешная загрузка каталога

Тест 1.2 Оформление заказа с одним товаром

Ожидаемый результат: Успешное создание заказа, уменьшение остатков, сохранение в файл.

Фактический результат: Заказ оформлен успешно (рис. 25)

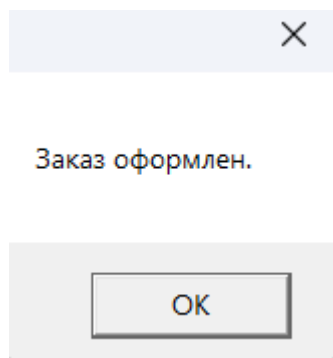


Рис. 25. Успешное оформление заказа

Тест 1.3 Создание поставки

Ожидаемый результат: Успешное пополнение товаров, сохранение в файл.

Фактический результат: Поставка создана успешно (рис. 26)

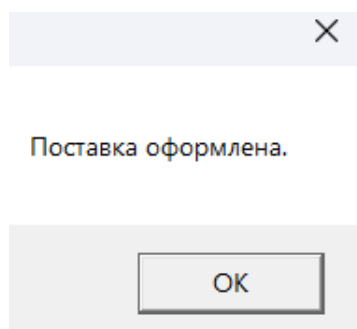


Рис. 26. Успешное оформление поставки

Тестирование в граничных условиях

Тест 2.1 Продажа максимально доступного количества товара

Ожидаемый результат: Продажа всего остатка товара.

Фактический результат: Продажа выполнена, остаток равен 0 (рис. 27)

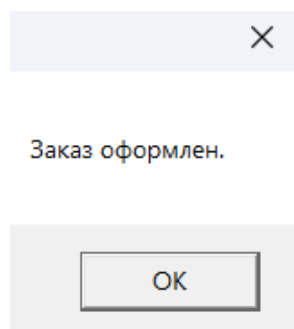


Рис. 27. Успешное оформление заказа

Тест 2.2 Поставка максимального разрешенного количества (100 единиц)

Ожидаемый результат: Успешное добавление 100 единиц товара.

Фактический результат: Поставка выполнена (рис. 28)

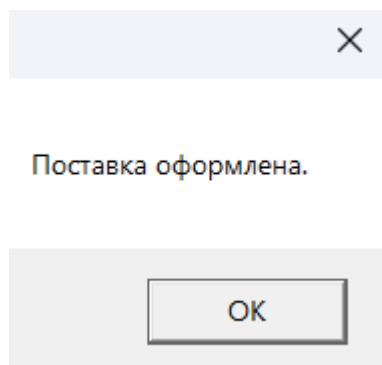


Рис. 28. Успешное оформление поставки

Тест 2.3 Поставка минимального количества (1 единица)

Ожидаемый результат: Успешное добавление 1 единицы товара.

Фактический результат: Поставка выполнена (рис. 29)

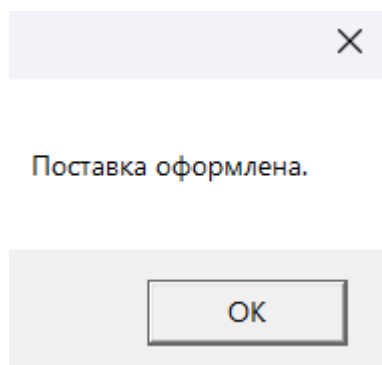


Рис. 29. Успешное оформление поставки

Тестирование в исключительных условиях

Тест 3.1 Попытка продажи при недостаточном количестве товара

Ожидаемый результат: Сообщение об ошибке, продажа не выполняется.

Фактический результат: Выведено сообщение об ошибке (рис. 30)

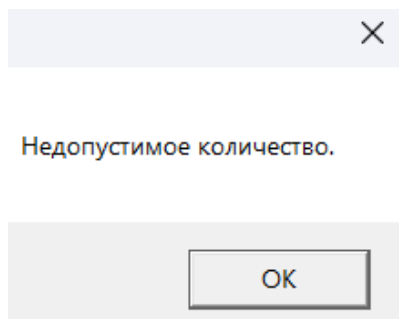


Рис. 30. Сообщение об ошибке

Тест 3.2 Ввод количества 0 при поставке

Ожидаемый результат: Сообщение об ошибке.

Фактический результат: Выведено сообщение об ошибке (рис. 31)

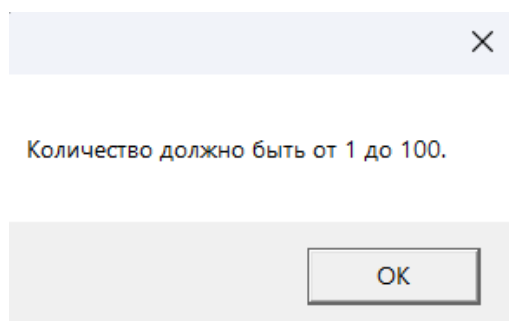


Рис. 31. Сообщение об ошибке

Тест 3.3 Ввод отрицательной или нулевой цены

Ожидаемый результат: Сообщение об ошибке.

Фактический результат: Выведено сообщение об ошибке (рис. 32)

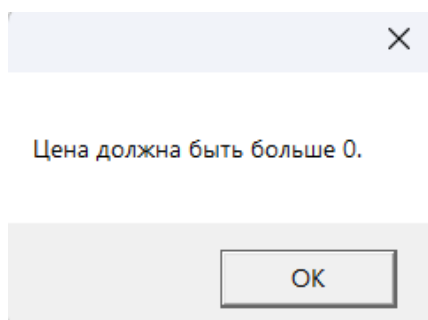


Рис. 32. Сообщение об ошибке

Результаты тестирования приведены в таблице

Табл. 8

Результаты тестирования программы

Номер теста	Условия тестирования	Результат
1.1	Нормальные	Успешно
1.2	Нормальные	Успешно
1.3	Нормальные	Успешно
2.1	Граничные	Успешно
2.2	Граничные	Успешно
2.3	Граничные	Успешно
3.1	Исключительные	Успешно
3.2	Исключительные	Успешно
3.3	Исключительные	Успешно

5. Сравнительный анализ реализаций

Это две реализации одной и той же системы управления музыкальным магазином. Обе программы предоставляют идентичный набор функций: управление каталогом товаров, обработку продаж, регистрацию поставок и изменение цен товаров. Основное отличие – платформа и тип интерфейса.

C++ версия представляет собой консольное приложение с текстовым меню, выполняющееся в терминале или командной строке. Взаимодействие с пользователем происходит через последовательность текстовых команд и ввода данных. C# реализация построена на платформе WPF и предоставляет полноценный графический интерфейс с отдельными окнами для каждой функции, кнопками и текстовыми полями, что делает программу более интуитивно понятной для рядового пользователя.

Архитектура и объектная модель. В C++ коде используется продвинутая объектная модель с наследованием: базовый класс Product и пять производных классов (Guitar, Keyboard, Wind, Percussion, Accessory). Это демонстрирует принципы ООП, но усложняет архитектуру. C# реализация использует упрощенную модель с одним классом Product, где категория хранится как строковое поле. Это делает код проще, но менее гибким для расширения.

Управление памятью и ресурсами. В C++ коде используется явное управление памятью: разработчик самостоятельно вызывает операторы new и delete, контролирует жизненный цикл объектов через конструкторы копирования/перемещения и деструкторы, отвечает за своевременное освобождение ресурсов. C# реализация полагается на автоматическое управление памятью через сборщик мусора, который самостоятельно освобождает неиспользуемые объекты, что снижает риск утечек памяти, но может влиять на производительность.

Работа с данными и сохранение состояния. C++ приложение имеет более полную систему сохранения данных: отдельные файлы для каталога, продаж, поставок и метаданных (ID счетчиков). C# программа сохраняет только каталог,

продажи и поставки, но не сохраняет текущие ID, которые сбрасываются при перезапуске программы.

Валидация входных данных. C++ версия содержит расширенную валидацию: проверку типа оплаты, валидацию номера карты (16 цифр), более детальную проверку бизнес-правил. C# реализация использует базовую валидацию через `PreviewTextInput` для числовых полей и простые проверки диапазонов.

Для практической системы управления музыкальным магазином, которая должна быть надежной, удобной для пользователей, легко расширяемой и поддерживаемой, C# с WPF является оптимальным выбором для фронтенд-части благодаря удобному графическому интерфейсу. Однако C++ версия имеет преимущества в надежности и полноте функционала для бизнес-логики. Идеальным решением было бы использование C# для интерфейса и C++ для ядра системы.

Заключение

В ходе выполнения работы была разработана и реализована система управления музыкальным магазином на двух языках программирования: C++ и C#. Обе реализации демонстрируют применение объектно-ориентированного подхода, но с разной степенью сложности архитектуры, что соответствует современным требованиям технологий программирования.

Достигнутые результаты.

Успешно разработаны две функционально идентичные программы, решающие задачу управления музыкальным магазином: C++ консольное приложение с текстовым интерфейсом и C# приложение с графическим интерфейсом WPF.

Создана архитектура управления данными с классами Product, Sale, Supply и FileManager в обеих реализациях, обеспечивающая целостность данных и сохранение состояния между сеансами работы.

Реализован полный набор функций, предусмотренных техническим заданием: просмотр каталога, оформление заказов (продаж), создание поставок, изменение цен товаров, формирование отчетов, сохранение данных в файлы.

Проведено комплексное тестирование обеих реализаций в нормальных, граничных и исключительных условиях, подтвердившее работоспособность программ и корректность обработки ошибок ввода.

Недостатки программ и пути возможного улучшения:

- Ограниченный функционал заказа. В обеих версиях нельзя добавить несколько разных товаров в один заказ. Решение: реализовать корзину товаров с возможностью добавления/удаления позиций перед оформлением заказа.
- Отсутствие истории операций в интерфейсе. Пользователь не видит историю продаж и поставок без просмотра файлов. Решение: добавить окно просмотра истории операций с фильтрацией и поиском.

- Упрощенная модель товаров в C#. Отсутствует наследование для разных категорий товаров. Решение: реализовать иерархию классов аналогично C++ версии для лучшей расширяемости.
- Нет расчета суммы заказа в C#. В графической версии не отображается общая стоимость заказа. Решение: добавить автоматический расчет и отображение суммы при добавлении товаров.

Вывод:

Работа успешно выполнена в соответствии с поставленным заданием. Доказана возможность реализации одной задачи на разных языках программирования с сохранением идентичной функциональности, но с разными подходами к архитектуре и интерфейсу. C++ версия демонстрирует более продвинутые техники ООП и управление памятью, в то время как C# версия предлагает более удобный пользовательский интерфейс и быструю разработку. Обе реализации имеют свои сильные стороны и могут быть улучшены за счет взаимного заимствования лучших практик.